

■ IndustrialIQ

AI-Powered Industrial Knowledge Intelligence Platform

Transforming unstructured plant data into actionable intelligence using multi-agent AI, knowledge graphs, and hybrid retrieval.

Team Member	Role	Responsibilities
Vaibhav	Data & Storage	Ingestion pipeline, Parsers, Neo4j schema & writer
Mohit	Intelligence & Application	LangGraph Supervisor, Hybrid Retrieval, Reranker, Eval
Kavyansh	UI + Compliance	Next.js Frontend, Compliance Agent, Lessons Learned Agent

GitHub: github.com/Mohit20198/ETH

SECTION 01

Problem Statement

Industrial plants and manufacturing facilities generate vast amounts of unstructured and siloed data — P&IDs, maintenance logs, safety inspection reports, compliance manuals (OSHA, OISD), and incident records. This data lives in disconnected PDFs, spreadsheets, and legacy databases.

■ Prolonged Downtime	Engineers waste hours during Root Cause Analysis (RCA) searching disconnected documents.
■ Compliance Risk	Manual verification of OSHA/OISD regulations is error-prone and increases audit failures.
■ Recurring Failures	Lessons learned from past incidents are buried in old reports, causing the same failures to repeat.
■ Knowledge Loss	Institutional knowledge leaves with experienced operators — no structured way to access it.

SECTION 02

Our Solution: IndustrialIQ

IndustrialIQ is a **multi-modal, multi-agent AI platform** that ingests industrial documents of any format, extracts structured knowledge into a Neo4j graph database, and enables engineers to query it in natural language — receiving accurate, cited, and confidence-scored answers in seconds.

Key Capabilities:

■ Multi-Modal Ingestion

Processes PDF, DOCX, XLSX, TXT, PNG/JPG (P&IDs;) and EML in a unified pipeline with SHA-256 deduplication.

■ Vision & OCR Engine

GPT-4 Vision + Tesseract OCR extracts text and spatial relationships from engineering diagrams.

■ Knowledge Graph

Entities and relationships extracted with two-pass LLM verification, written to Neo4j with full provenance.

■ Hybrid Retrieval

ChromaDB semantic vector search combined with Neo4j graph traversal and a weighted reranker (0.6/0.4).

■ Multi-Agent Orchestration

LangGraph supervisor dispatches to Copilot, Maintenance, Compliance, and Lessons Learned agents in parallel.

■ Traceability

Every answer is backed by source citations, document names, and a calibrated confidence score.

SECTION 03

System Architecture

Query Pipeline

```
User Natural Language Query
|
v
FastAPI Gateway (backend/api/main.py)
|
v
LangGraph Supervisor (backend/agents/supervisor.py)
|
+-- [1] classify_node --> Query Type: single-fact / multi-hop / aggregation
|
+-- [2] retrieve_node +-- ChromaDB Vector Search (always runs)
|   +-- Neo4j Graph Traversal (multi-hop fallback)
|   +--> Weighted Reranker (vector=0.6, graph=0.4)
|
+-- [3] dispatch_node --> Parallel Specialist Agents:
|   +-- CopilotAgent (general Q&A;, summarization)
|   +-- MaintenanceAgent (RCA, MTBF, work orders)
|   +-- ComplianceAgent (OSHA, OISD, Factory Act)
|   +-- LessonsLearnedAgent (failure patterns, incidents)
|
+-- [4] synthesize_node --> Final answer + deduplicated citations + confidence
|
v
AgentResponse { answer, citations, confidence, graph_path, latency_ms }
```

Figure 1: End-to-end query pipeline through the LangGraph supervisor

Document Ingestion Pipeline

```
New Document Upload (PDF / DOCX / XLSX / TXT / PNG / JPG / EML)
|
+-- SHA-256 Fingerprint Check --> Already seen? SKIP
|
+-- Parser Router:
|   PDF --> pdfplumber + OCR fallback (Tesseract)
|   P&ID/IMG --> OpenCV preprocess + pytesseract + GPT-4 Vision
|   Excel --> openpyxl (spatial layout preserved)
|   DOCX --> python-docx
|   TXT/EML --> direct read
|
+-- Semantic Chunking (per doc type)
+-- Embed chunks --> ChromaDB (text-embedding-3-small)
|
+-- Two-Pass LLM Extraction --> Neo4j Knowledge Graph
Pass 1: LLM proposes { nodes, edges } against pinned ontology
Pass 2: Verifier assigns confidence, rejects < 0.70
Only verified facts written (with source_doc_id + audit log)
```

Figure 2: Document ingestion pipeline with two-pass verification

SECTION 04

Technology Stack

Layer	Technology	Purpose
Agent Orchestration	LangGraph, LangChain	Stateful multi-agent workflow with conditional routing
Backend API	FastAPI (Python)	Async REST API with auto-generated OpenAPI docs
LLM / Vision	OpenAI GPT-4o	Reasoning, extraction, Cypher generation, vision OCR
Vector Store	ChromaDB	Semantic similarity search on document chunks
Graph Database	Neo4j AuraDB	Multi-hop relationship traversal via Cypher
Embeddings	text-embedding-3-small (OpenAI)	Document chunk embeddings for vector search
OCR / Vision	Tesseract OCR, OpenCV, pdf2image	P&ID diagram text and structure extraction
Frontend	Next.js 16, TailwindCSS, TypeScript	Real-time chat UI with citations and graph panel
Evaluation	Ragas, Langfuse	Answer relevancy, faithfulness, latency metrics
Deployment	Render (Docker), Vercel	Backend on Render, Frontend on Vercel (free tier)

SECTION 05

Knowledge Graph Ontology

The shared ontology (backend/shared/ontology.py) is a locked contract. All edges with confidence below 0.70 are rejected and never written to the graph.

Node Types	Equipment, Document, Parameter, Person, Regulation, Event
Edge Types	HAS_PARAMETER, DOCUMENTED_IN, PERFORMED_BY, PRECEDED_BY, REGULATED_BY, CONNECTED_TO, MENTIONS, CAUSED_BY
Provenance	source_doc_id, confidence, extraction_pass_id, extracted_at, extracted_by, verified, role_tag, raw_text_span
Threshold	Only facts with confidence ≥ 0.70 are written to Neo4j

SECTION 06

Suggested Demo Queries for Judges

The repository includes a pre-ingested ChromaDB vector store with 10 sample industrial documents. No re-ingestion required — just run the backend and try these queries:

#	Category	Sample Query
1	Maintenance & RCA	What caused the last failure of Pump P-101 and what is its MTBF?
2	Compliance	Are we compliant with OSHA gas detector calibration requirements?
3	Multi-Hop Graph	Show maintenance history for all equipment connected to Valve V-201.
4	Lessons Learned	Are there recurring failure patterns in our blast furnace equipment?

5	Vision / P&ID	What are the operating parameters shown in the P&ID diagram?
---	---------------	--

SECTION 07

Deployment

Service	Provider	Notes
Frontend (Next.js)	Vercel	Deployed from /frontend directory, auto-builds on git push
Backend API (FastAPI)	Render	Docker-based deploy using Dockerfile + render.yaml
Graph Database	Neo4j AuraDB	Free cloud instance, bolt:// URI set as env var on Render
Source Code	GitHub	github.com/Mohit20198/ETH (includes pre-ingested ChromaDB data)